Report

Muhammad Ali Sher, 23i-0683, Section-F

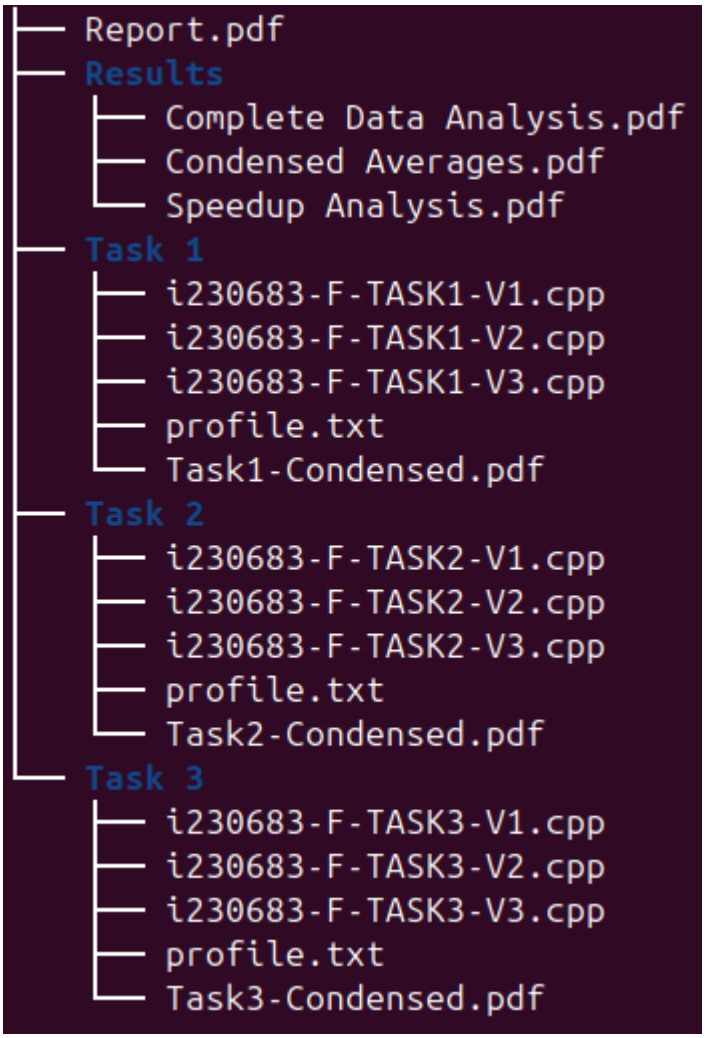
Repository: https://github.com/Ali-Sher12/Semester-6-Assignments-and-Projects

1. Machine Specifications

- CPU model: Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz
- Physical cores: 4
- Logical threads: 8
- RAM: 16GB DDR4
- OS version: Ubuntu 24.04.3 LTS

2. Contents of Submission

The zip folder contains the following contents:



- Prof.txt is the profiling done using perf.

- 'Complete Data Analysis' contains all thread values against each data percentage for each version of each task, having 3 trials.

- 'Condensed Averages' contains the averages across each version of each task and then across each task. (Average of 3 trials), (Only for 100% data).

- 'Speedup Analysis' contains the data comparing the speedup of V2 and V3 against V1 for each value of thread. (Average of 3 trials), (Only for 100% data), (Graph included).

- 'Task-N Condensed.pdf ' contains all thread values for each version of each task, having 3 trials for that Task only. (Only for 100% data), (Graph included).

3. Implementation Summary

- Task 1
    - V1

Started by creating a data_ struct, which held an entire row of the dataset (y,y', and all x values). Read the data into a global vector using fstream and sstream. All the data metrics were globals (TN, FN, etc). The time was recorded through `clock_gettime()` as recommended. Data was read once, and everything else was in a loop for all the data percentages.

    - V2 & V3

I used a double loop in the main, so that I could have 3 data percentages (10, 50, 100) for each iteration using 1,2,4 or 8 number of threads. I divided the data into chunks via index manipulation. A mistake I made early on was using mutexes within the loop of the computation, as it increased the execution time. I fixed that by having local variables within the loop and adding them into the actual variables outside using a mutex.

For V3, I added these lines after pthread_create() to manually assign each thread to a core:

```
cpu_set_t cpuset;

CPU_ZERO(&cpuset);

CPU_SET(i%no_of_cores, &cpuset);

pthread_setaffinity_np(threads[i], sizeof(cpu_set_t), &cpuset);
```

These lines were added in V3, in each task. Being the only difference from V2.

- Task 2
    - V1

A mistake I made early on was using a 2D float array, without realizing that the size of the Matrix would go in Terabytes. So I redid the entire task. I used a struct the held row, column, and value for each non-zero row. In addition, the data reading was done in such a way that each missing value was assumed to be 1. I had to change the logic of my computation as well, as earlier I had opted for a simple 2D loop, then flattened it for V2 and V3. This time, the computation loop was simple. Rest(time measuring, etc.) was the same as the previous V1.

    - V2 & V3

Essentially the same as in V2 of Task 1. Chunk division and mutexes outside the computation loop to copy data from the local variable.

- Task 3
    - V1

The computation function had three main parts: BFS, finding the maximum distance, and finding the number of nodes at level k. Instead of using a vector this time, I decided to use a map to sacrifice time for space. Also, this time, data reading was done within the loop to properly set data percentages. Rest was the same.

    - V2 & V3

This proved a bit tricky as I had no clue how to parallelize BFS. So I decided to parallelize, finding maximum distances and finding nodes at ‘k’ distance. This did not have a huge speedup, unfortunately. Rest was the same.

4. Correctness Validation

- How V2/V3 validated against V1

  All data was cross-checked, and there were no significant differences across all versions. The data was cross-checked manually by me. And is present on my GitHub(link provided) in case it needs to be rechecked.